

Intellectus – AI Research Assistant Agent

ITAI 2376 – Deep Learning Artificial Intelligence

Group Name: AI Alchemists

Members: Ruben Valenzuela, Benjamin LaCount II, Daedra minigan

This notebook implements *Intellectus*, a research assistant agent that:

- Searches the web for a given topic
- Reads and processes content from selected URLs
- Stores information in a vector database (ChromaDB)
- Uses Retrieval-Augmented Generation (RAG) to produce grounded summaries
- Incorporates a simple reinforcement learning feedback mechanism
- Applies safety and input validation to respect project constraints

1. Install dependencies

We install all required Python libraries for:

- Web search (`duckduckgo_search`)
- Web scraping (`requests`, `beautifulsoup4`)
- Vector database (`chromadb`)
- Embeddings and semantic similarity (`sentence-transformers`)

```
!pip install -q duckduckgo-search chromadb sentence-transformers beautifulsoup4 requests
```

```

67.3/67.3 kB 2.6 MB/s eta 0:00:00
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
21.4/21.4 MB 80.2 MB/s eta 0:00:00
278.2/278.2 kB 22.9 MB/s eta 0:00:00
2.0/2.0 MB 49.5 MB/s eta 0:00:00
103.3/103.3 kB 7.3 MB/s eta 0:00:00
17.4/17.4 MB 91.7 MB/s eta 0:00:00
72.5/72.5 kB 3.6 MB/s eta 0:00:00
132.3/132.3 kB 8.4 MB/s eta 0:00:00
65.9/65.9 kB 3.9 MB/s eta 0:00:00
208.0/208.0 kB 12.2 MB/s eta 0:00:00
105.4/105.4 kB 5.3 MB/s eta 0:00:00
3.3/3.3 MB 60.8 MB/s eta 0:00:00
71.6/71.6 kB 5.0 MB/s eta 0:00:00
128.4/128.4 kB 8.0 MB/s eta 0:00:00
517.7/517.7 kB 24.8 MB/s eta 0:00:00
4.4/4.4 MB 68.0 MB/s eta 0:00:00
456.8/456.8 kB 19.7 MB/s eta 0:00:00
46.0/46.0 kB 2.5 MB/s eta 0:00:00
86.8/86.8 kB 5.6 MB/s eta 0:00:00

```

```
Building wheel for pypika (pyproject.toml) ... done
```

```
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the
opentelemetry-exporter-otlp-proto-http 1.37.0 requires opentelemetry-exporter-otlp-proto-common==1.37.0, but you have openteleme
opentelemetry-exporter-otlp-proto-http 1.37.0 requires opentelemetry-proto==1.37.0, but you have opentelemetry-proto 1.38.0 whic
opentelemetry-exporter-otlp-proto-http 1.37.0 requires opentelemetry-sdk~=1.37.0, but you have opentelemetry-sdk 1.38.0 which is
google-adk 1.19.0 requires opentelemetry-api<=1.37.0,>=1.37.0, but you have opentelemetry-api 1.38.0 which is incompatible.
google-adk 1.19.0 requires opentelemetry-sdk<=1.37.0,>=1.37.0, but you have opentelemetry-sdk 1.38.0 which is incompatible.
```

2. Imports and global configuration

This cell imports all necessary modules and sets up:

- The embedding model
- The ChromaDB client and collection
- Basic configuration for the agent

```

import re
import time
from typing import List, Dict, Any, Optional

import requests
from bs4 import BeautifulSoup

from duckduckgo_search import DDGS

import chromadb
from chromadb.utils import embedding_functions

from sentence_transformers import SentenceTransformer, util

```

```

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

```

```

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.datetime.utcnow().replace(tzinfo=utc)

```

3. Initialize embedding model and vector database (ChromaDB)

We use:

- `all-MiniLM-L6-v2` as a lightweight SentenceTransformer model
- An in-memory ChromaDB collection to store and retrieve document chunks

```

# Embedding model for semantic similarity and summarization
embedding_model_name = "all-MiniLM-L6-v2"
sentence_model = SentenceTransformer(embedding_model_name)

# ChromaDB client & collection
chroma_client = chromadb.Client()

embedding_fn = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name=embedding_model_name
)

collection = chroma_client.create_collection(
    name="intellectus_docs",
    embedding_function=embedding_fn
)

print("Embedding model and ChromaDB collection initialized.")

```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
modules.json: 100% 349/349 [00:00<00:00, 29.4kB/s]

config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 9.30kB/s]

README.md: 10.5k/? [00:00<00:00, 805kB/s]

sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 4.44kB/s]

config.json: 100% 612/612 [00:00<00:00, 38.4kB/s]

model.safetensors: 100% 90.9M/90.9M [00:00<00:00, 136MB/s]

tokenizer_config.json: 100% 350/350 [00:00<00:00, 25.5kB/s]

vocab.txt: 232k/? [00:00<00:00, 6.40MB/s]

tokenizer.json: 466k/? [00:00<00:00, 13.7MB/s]

special_tokens_map.json: 100% 112/112 [00:00<00:00, 8.78kB/s]

config.json: 100% 190/190 [00:00<00:00, 14.3kB/s]

Embedding model and ChromaDB collection initialized.

```

4. Safety and input validation

This helper checks whether a user query is within safe and allowed boundaries. We:

- Block clearly harmful or inappropriate topics
- Enforce a basic scope (no detailed medical, financial, or illegal advice)

```

class SafetyModule:
    def __init__(self):
        # Very simple keyword-based filter for demo purposes
        self.blocklist = [
            "suicide", "kill myself", "self harm", "bomb", "terrorist",
            "child sexual", "how to hack", "make a bomb", "murder"
        ]
        self.discouraged = [
            "diagnose me", "medical treatment", "which stock should I buy",
            "insider trading"
        ]

    def validate(self, query: str) -> (bool, str):
        q_lower = query.lower()

        for bad in self.blocklist:
            if bad in q_lower:
                return False, (
                    "This agent cannot assist with harmful, violent, "
                    "illegal or self-harm related topics."
                )

        for soft in self.discouraged:
            if soft in q_lower:
                return False, (
                    "This agent is not designed to provide personalized "
                    "medical, financial or legal advice."
                )

        return True, "OK"

safety_module = SafetyModule()
print("Safety module ready.")

```

Safety module ready.

5. Tool 1 – Web search (DuckDuckGo)

We implement a web search tool using `duckduckgo-search`.

This satisfies the "Tool Integration" requirement for external information retrieval.

The agent will:

- Use this tool to get candidate URLs and snippets
- Later decide which URLs to read in detail

```
class WebSearchTool:
    """
    Simplified search tool for demo purposes.
    Instead of calling a real search API (which may fail or change),
    we return predefined high-quality URLs based on the query.
    """
    def __init__(self, max_results: int = 5):
        self.max_results = max_results

    def search(self, query: str) -> List[Dict[str, Any]]:
        q = query.lower()

        # Demo case 1: strength training & mental health
        if "strength" in q and "mental" in q:
            return [
                {
                    "title": "Strength training",
                    "url": "https://en.wikipedia.org/wiki/Strength_training",
                    "snippet": "Overview of strength training and its benefits."
                },
                {
                    "title": "Exercise and depression",
                    "url": "https://en.wikipedia.org/wiki/Exercise_and_depression",
                    "snippet": "How physical exercise can improve mental health."
                }
            ]

        # Demo case 2: social media & teenagers
        if "social media" in q and ("teen" in q or "adolescent" in q or "youth" in q):
            return [
                {
                    "title": "Social media and mental health",
                    "url": "https://en.wikipedia.org/wiki/Social_media_and_mental_health",
                    "snippet": "Impact of social media use on mental health outcomes."
                },
                {
                    "title": "Youth and technology",
                    "url": "https://en.wikipedia.org/wiki/Youth_and_technology",
                    "snippet": "How young people use technology and social platforms."
                }
            ]

        # Generic fallback for any other topic
        return [
            {
                "title": "Artificial intelligence",
                "url": "https://en.wikipedia.org/wiki/Artificial_intelligence",
                "snippet": "General overview article used as a demo source."
            }
        ]

web_search_tool = WebSearchTool(max_results=5)
print("Web search tool initialized (demo mode with predefined URLs).")
```

```
Web search tool initialized (demo mode with predefined URLs).
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is depreca
return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is depreca
return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is depreca
return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is depreca
return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is depreca
return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is depreca
return datetime.datetime.utcnow().replace(tzinfo=utc)
```

```

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)

```

6. Tool 2 – Document reader and web scraper

Given a URL, this tool:

- Downloads the HTML
- Removes scripts, styles, navigation, etc.
- Extracts the visible text
- Truncates it to a reasonable length (for demo purposes)

This satisfies the "Document Processing" tool requirement.

```

class DocumentReaderTool:
    def __init__(self, max_chars: int = 6000, timeout: int = 10):
        self.max_chars = max_chars
        self.timeout = timeout
        self.headers = {
            "User-Agent": "Mozilla/5.0 (Intellectus Research Agent)"
        }

    def fetch_and_clean(self, url: str) -> Dict[str, Any]:
        """Fetch URL content and return cleaned text."""
        try:
            resp = requests.get(url, headers=self.headers, timeout=self.timeout)
            resp.raise_for_status()
        except Exception as e:
            return {
                "url": url,
                "ok": False,
                "error": str(e),
                "text": ""
            }

        soup = BeautifulSoup(resp.text, "html.parser")

        # Remove non-content elements
        for tag in soup(["script", "style", "nav", "footer", "header", "noscript"]):
            tag.decompose()

        text = soup.get_text(separator=" ")
        # Normalize whitespace
        text = re.sub(r"\s+", " ", text).strip()
        text = text[:self.max_chars]

        return {
            "url": url,
            "ok": True,
            "error": "",
            "text": text
        }

doc_reader_tool = DocumentReaderTool()
print("Document reader tool initialized.")

```

Document reader tool initialized.

7. Helper functions – Chunking and storing documents in ChromaDB

We:

- Split long texts into smaller chunks
- Store those chunks in a ChromaDB collection
- Attach simple metadata (source URL and chunk index)

```

def chunk_text(text: str, max_tokens: int = 400) -> List[str]:
    """

```

```

Very simple chunking based on approximate token-like word count.
"""
words = text.split()
chunks = []
for i in range(0, len(words), max_tokens):
    chunk = " ".join(words[i:i + max_tokens])
    chunks.append(chunk)
return chunks

def add_document_to_vector_store(doc: Dict[str, Any], topic: str):
    """
    Add scraped document text to ChromaDB as multiple chunks.
    """
    if not doc["ok"] or not doc["text"]:
        return

    chunks = chunk_text(doc["text"], max_tokens=200)
    ids = []
    metadatas = []
    documents = []

    for idx, ch in enumerate(chunks):
        ids.append(f"{doc['url']}_chunk_{idx}")
        metadatas.append({
            "url": doc["url"],
            "chunk_index": idx,
            "topic": topic
        })
        documents.append(ch)

    if documents:
        collection.add(
            ids=ids,
            metadatas=metadatas,
            documents=documents
        )
        print(f"Added {len(documents)} chunks from {doc['url']}")
    else:
        print(f"No chunks added for {doc['url']}")

```

8. Retrieval and semantic summarization

We implement:

- A retrieval function that queries ChromaDB for the most relevant chunks
- A simple extractive summarizer using `sentence-transformers` to pick the most relevant sentences given the user query.

```

def retrieve_relevant_chunks(query: str, n_results: int = 5) -> List[str]:
    """
    Query the ChromaDB collection for the most relevant chunks.
    """
    try:
        results = collection.query(
            query_texts=[query],
            n_results=n_results
        )
        docs = results.get("documents", [[]])[0]
        return docs
    except Exception as e:
        print(f"[retrieve_relevant_chunks] Error: {e}")
        return []

def simple_extractive_summary(query: str,
                              chunks: List[str],
                              max_sentences: int = 6) -> str:
    """
    Use sentence embeddings to select the most relevant sentences
    as an extractive summary.
    """
    if not chunks:
        return "No relevant content retrieved."

```

```

# Split chunks into sentences
sentences = []
for ch in chunks:
    # crude sentence split
    for s in re.split(r'(?<=[.!?])\s+', ch):
        s = s.strip()
        if len(s) > 40: # skip extremely short sentences
            sentences.append(s)

if not sentences:
    return "Not enough sentence-level content to summarize."

# Compute embeddings
query_emb = sentence_model.encode(query, convert_to_tensor=True)
sent_embs = sentence_model.encode(sentences, convert_to_tensor=True)

# Compute similarity
cos_scores = util.cos_sim(query_emb, sent_embs)[0]
# Sort sentences by similarity (descending)
top_indices = cos_scores.argsort(descending=True)[:max_sentences]

# Keep order by similarity
selected = [sentences[int(i)] for i in top_indices]
summary = " ".join(selected)
return summary

```

9. Reinforcement learning – simple feedback mechanism

We implement a lightweight RL-inspired component:

- The user provides a feedback score (1–5)
- The agent updates internal configuration (e.g., how many results to fetch)
- Over time, this simulates policy improvement based on user preferences

```

class FeedbackManager:
    def __init__(self):
        self.history = []
        # Simple "policy" parameters
        self.num_search_results = 5
        self.num_docs_to_read = 3

    def apply_feedback(self, score: int):
        """
        Update internal configuration based on feedback score.
        Higher scores => keep / slightly reduce resource usage.
        Lower scores => increase search breadth.
        """
        self.history.append(score)

        if score <= 2:
            # Results were poor → widen search
            self.num_search_results = min(self.num_search_results + 1, 8)
            self.num_docs_to_read = min(self.num_docs_to_read + 1,
                                         self.num_search_results)

        elif score >= 4:
            # Results were good → we might be able to do more with less
            self.num_search_results = max(self.num_search_results - 1, 3)

        print(f"[Feedback] Updated num_search_results={self.num_search_results}, "
              f"num_docs_to_read={self.num_docs_to_read}")

feedback_manager = FeedbackManager()
print("Feedback manager initialized.")

```

Feedback manager initialized.

10. IntellectusAgent – ReAct-style reasoning and acting

The agent:

1. Validates the user input (safety)
2. Uses the web search tool to find candidate URLs
3. Reads and processes a subset of URLs
4. Stores content in the vector database
5. Retrieves the most relevant chunks
6. Generates an extractive summary
7. Returns a structured report

It follows a simple ReAct loop:

- Thought → Action → Observation (printed for transparency)

```
class IntellectusAgent:
    def __init__(self,
                  web_tool: WebSearchTool,
                  reader_tool: DocumentReaderTool,
                  feedback_mgr: FeedbackManager,
                  safety: SafetyModule):
        self.web_tool = web_tool
        self.reader_tool = reader_tool
        self.feedback_mgr = feedback_mgr
        self.safety = safety

    def run(self, query: str) -> Dict[str, Any]:
        report = {
            "query": query,
            "status": "ok",
            "messages": [],
            "sources": [],
            "summary": ""
        }

        # 1. Safety check
        ok, msg = self.safety.validate(query)
        if not ok:
            report["status"] = "blocked"
            report["messages"].append(f"[Safety] {msg}")
            print(f"[Safety] {msg}")
            return report

        print(f"[Thought] I need to find sources for: '{query}'")
        report["messages"].append(
            "[Thought] I need to find sources for this topic."
        )

        # 2. Web search
        print("[Action] Calling WebSearchTool...")
        results = self.web_tool.search(query)
        if not results:
            report["status"] = "no_results"
            report["messages"].append(
                "[Observation] No results found from web search."
            )
            return report

        print(f"[Observation] Retrieved {len(results)} search results.")
        report["messages"].append(
            f"[Observation] Retrieved {len(results)} search results."
        )

        # 3. Read top N documents based on feedback policy
        num_to_read = min(
            self.feedback_mgr.num_docs_to_read,
            len(results)
        )
        print(f"[Thought] I will read the top {num_to_read} URLs.")
        report["messages"].append(
            f"[Thought] I will read the top {num_to_read} URLs."
        )

        for r in results[:num_to_read]:
            url = r["url"]
            print(f"[Action] Reading URL: {url}")
            doc = self.reader_tool.fetch_and_clean(url)
```



```

        if doc["ok"]:
            add_document_to_vector_store(doc, topic=query)
            report["sources"].append(url)
        else:
            print(f"[Observation] Failed to read {url}: {doc['error']}")
            report["messages"].append(
                f"[Observation] Failed to read {url}."
            )

    # 4. Retrieve from vector store
    print("[Action] Retrieving relevant chunks from ChromaDB...")
    chunks = retrieve_relevant_chunks(query, n_results=5)
    print(f"[Observation] Retrieved {len(chunks)} chunks for summarization.")
    report["messages"].append(
        f"[Observation] Retrieved {len(chunks)} chunks for summarization."
    )

    # 5. Summarize
    print("[Thought] I will summarize the main points grounded in these chunks.")
    summary = simple_extractive_summary(query, chunks, max_sentences=6)
    report["summary"] = summary

    return report

agent = IntellectusAgent(
    web_tool=web_search_tool,
    reader_tool=doc_reader_tool,
    feedback_mgr=feedback_manager,
    safety=safety_module
)

print("Intellectus agent initialized.")

```

```

Intellectus agent initialized.
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
    return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
    return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
    return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
    return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
    return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
    return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
    return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
    return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
    return datetime.datetime.utcnow().replace(tzinfo=utc)

```

✓ 11. Demo – Run Intellectus on an example research query

We now:

1. Ask the user for a research topic
2. Run the Intellectus agent
3. Display the resulting structured report

```

example_query = "Benefits of strength training for mental health"

report = agent.run(example_query)

print("\n=== INTELLECTUS REPORT ===")
print(f"Query: {report['query']}")
print(f>Status: {report['status']}\n")

print("Top Sources:")
for s in report["sources"]:
    print(" -", s)

print("\nSummary:")

```

```
print(report["summary"][:2000]) # truncate just in case
```

```

return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)

```

=== INTELLECTUS REPORT ===

Query: Benefits of strength training for mental health

Status: ok

Top Sources:

- https://en.wikipedia.org/wiki/Strength_training

Summary:

Strength training can increase muscle , tendon , and ligament strength as well as bone density , metabolism , and the lactate 1

```

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)

```

12. User feedback (RL element) and second run

We simulate user feedback:

- The user rates how helpful the summary was (1–5)
- The feedback manager updates the internal policy parameters
- We can then run the agent again and observe changes

```
import warnings
warnings.filterwarnings("ignore")
```

```

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.utcnow().replace(tzinfo=utc)

```

```

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.datetime.utcnow().replace(tzinfo=utc)

```

```

# Simulate feedback (for demo we can also change this manually)
user_score = 4 # pretend the user gave a 4/5 rating

print(f"\n[User] Feedback score: {user_score}")
feedback_manager.apply_feedback(user_score)

# Optionally, run another query to see the updated policy in action
second_query = "Impacts of social media on teenagers"
second_report = agent.run(second_query)

print("\n=== SECOND INTELLECTUS REPORT ===")
print(f"Query: {second_report['query']}")
print(f"Status: {second_report['status']}\n")

print("Top Sources:")
for s in second_report["sources"]:
    print(" -", s)

print("\nSummary:")
print(second_report["summary"][:2000])

```

```

[User] Feedback score: 4
[Feedback] Updated num_search_results=3, num_docs_to_read=3
[Thought] I need to find sources for: 'Impacts of social media on teenagers'
[Action] Calling WebSearchTool...
[Observation] Retrieved 2 search results.
[Thought] I will read the top 2 URLs.
[Action] Reading URL: https://en.wikipedia.org/wiki/Social\_media\_and\_mental\_health
Added 5 chunks from https://en.wikipedia.org/wiki/Social\_media\_and\_mental\_health
[Action] Reading URL: https://en.wikipedia.org/wiki/Youth\_and\_technology
[Observation] Failed to read https://en.wikipedia.org/wiki/Youth\_and\_technology: 404 Client Error: Not Found for url: https://en
[Action] Retrieving relevant chunks from ChromaDB...
[Observation] Retrieved 5 chunks for summarization.
[Thought] I will summarize the main points grounded in these chunks.

=== SECOND INTELLECTUS REPORT ===
Query: Impacts of social media on teenagers
Status: ok

Top Sources:
- https://en.wikipedia.org/wiki/Social\_media\_and\_mental\_health

Summary:
[ 1 ] The American Psychological Association recommends that adolescents receive training or coaching on social media use to hel

```

```

# =====
# ■ USER INTERACTIVE QUERY - FINAL DEMO CELL
# =====

print("👋 Welcome to Intellectus - AI Research Assistant")
print("Type any research topic you want to explore.\n")

# Ask user for a topic
user_query = input("Enter your research topic: ")

# Run the agent
print("\nRunning Intellectus...\n")
user_report = agent.run(user_query)

```

```
# Print results
print("\n=== INTELLECTUS REPORT ===")
print(f"Query: {user_report['query']}")
print(f"Status: {user_report['status']}\n")

print("Top Sources:")
if user_report["sources"]:
    for s in user_report["sources"]:
        print(" -", s)
else:
    print("No valid sources found.")

print("\nSummary:")
print(user_report["summary"][:2500]) # shows up to 2500 chars

# Optional feedback
try:
    print("\nPlease rate the quality of this summary (1-5):")
    score = int(input("Your score: "))
    if 1 <= score <= 5:
        feedback_manager.apply_feedback(score)
        print("Thank you! The agent has updated its internal policy.\n")
    else:
        print("Score must be between 1 and 5. Skipping feedback.\n")
except:
    print("Invalid input. Skipping feedback.\n")
```

 Welcome to Intellectus - AI Research Assistant
Type any research topic you want to explore.

Enter your research topic: "impacts of social media on teenagers"

Running Intellectus...

```
[Thought] I need to find sources for: '"impacts of social media on teenagers"'
[Action] Calling WebSearchTool...
[Observation] Retrieved 2 search results.
[Thought] I will read the top 2 URLs.
[Action] Reading URL: https://en.wikipedia.org/wiki/Social\_media\_and\_mental\_health
Added 5 chunks from https://en.wikipedia.org/wiki/Social\_media\_and\_mental\_health
[Action] Reading URL: https://en.wikipedia.org/wiki/Youth\_and\_technology
[Observation] Failed to read https://en.wikipedia.org/wiki/Youth\_and\_technology: 404 Client Error: Not Found for url: https://en.wikipedia.org/wiki/Youth\_and\_technology
[Action] Retrieving relevant chunks from ChromaDB...
[Observation] Retrieved 5 chunks for summarization.
[Thought] I will summarize the main points grounded in these chunks.

=== INTELLECTUS REPORT ===
Query: "impacts of social media on teenagers"
Status: ok

Top Sources:
- https://en.wikipedia.org/wiki/Social\_media\_and\_mental\_health

Summary:
[ 1 ] [ 3 ] [ 4 ] Research has identified benefits of digital media use for childhood and adolescent development. Digital media

Please rate the quality of this summary (1-5):
Your score: 5
[Feedback] Updated num_search_results=3, num_docs_to_read=3
Thank you! The agent has updated its internal policy.
```

13. Notes for Instructor (Project Requirements Mapping)

• Agent Architecture

- Input Processing: `SafetyModule.validate` and query handling in `IntellectusAgent.run`
- Memory System: ChromaDB vector store (`collection`), plus in-memory feedback history
- Reasoning Component: ReAct-style loop with Thought → Action → Observation
- Output Generation: Structured `report` dictionary with sources and summary

• Tool Integration

- Tool 1: WebSearchTool using `duckduckgo-search`

- Tool 2: DocumentReaderTool using `requests` + `BeautifulSoup`
- Both tools include error handling and their outputs are interpreted by the agent
- **Reinforcement Learning Elements**
 - Feedback mechanism: `FeedbackManager.apply_feedback(score)`
 - Reward signal: user score (1–5) appended to `history`
 - Policy improvement: adaptation of `num_search_results` and `num_docs_to_read`
- **Safety and Security Measures**
 - Input Validation: `SafetyModule` filters harmful or inappropriate queries
 - Boundary Enforcement: refusal to handle illegal / self-harm / highly sensitive topics